



A genetic-based clustering algorithm for efficient resource allocating of IoT applications in layered fog heterogeneous platforms

Kimia Abedpour¹ · Mirsaeid Hosseini Shirvani² · Elmira Abedpour³

Received: 22 February 2023 / Revised: 27 March 2023 / Accepted: 12 April 2023 / Published online: 10 May 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract

Fog Computing paradigm that provisions low-latency computing services at the edge network, is a bonanza for supply chain computing resources in Internet of Things (IoT) applications. In different scenarios such as smart homes/healthcare systems, multiple IoT applications are distributed simultaneously in cloud and fog nodes to provide different IoT-based services. In addition, each program requires resources and has its desired quality of service (QoS) which should be met. One of the key challenges in fog computing environment is how to efficiently allocate resources to IoT applications because inefficient resource allocation leads to burdening providers high costs and it lowers down the delivered QoS to users. Since the majority of IoT applications are time-sensitive, the low delay and near physically allocated resources improve the amount of delivered QoS. Therefore, the resource clustering algorithms with the lowest distance error rate and the lowest delay as a consequence are favorable. The aim is to reduce clustering errors and improve the overall performance of the system. This paper formulates resource allocation to IoT applications in heterogeneous 4-layered fog platforms to an optimization problem. To solve this problem, a fusion approach incorporating a genetic algorithm (GA) and the k-means clustering approach is presented. Firstly, it utilizes the k-means approach and Jaccard measurement to cluster fog nodes with a minimum clustering rate. Then, the resources of fog clusters are allocated to IoT devices with the minimum error rate by incorporating GA algorithm. This selection of processing nodes in a fog layer helps to minimize latency and allows applications to access resources simultaneously. The simulation results in extensive scenarios prove the superiority of the proposed algorithm against other successful meta-heuristic approaches in terms of the objective function and lowest error/delay rate.

Keywords Fog computing · Internet of Things · k-means clustering method · Jaccard measurement · Genetic optimization algorithm

1 Introduction

In recent years, the Internet of Things (IoT) technology is being pervasively used to make human life easier. This technology has been introduced in various industries such

as the transportation industry [1], the medical industry [2], and home applications [3], etc. which involve a wide range of computing resource requests. Nowadays, the evolution of computing can be considered on the ground of an essential element in human life. There is also a requirement for people to be able to do their heavy computing works through services without having expensive hardware and software. In such a case, users try to access resources based on their needs, regardless of where and how the service is provided [4, 5]. There are various examples of computing systems that try to provide such services to users. Cluster computing, grid computing, cloud computing, and fog computing are some of these systems. Fog Computing, provides low-latency computing services at the edge of the network which is inherently heterogeneous, an activator for

✉ Mirsaeid Hosseini Shirvani
mirsaeid_hosseini@iausari.ac.ir;
mirsaeid_hosseini@yahoo.com

¹ Department of Computer Engineering, Tabarestan Chalus Institute, Chalus, Iran

² Department of Computer Engineering, Sari Branch, Islamic Azad University, Sari, Iran

³ Interuniversity Department of Territorial Science, Project and Politics, Polytechnic, University of Turin, Turin, Italy

next-generation IoT systems. With the emergence of IoT applications and their pervasiveness, except for fog computing other computing paradigms suffer from numerous shortcomings such as geographical limitations and non-flexibility tailored with resource usage patterns which cannot support these kinds of applications in most cases. One of the most important challenges in IoT applications is relevant to the diversity and their diverse resource requests as well. Therefore, the resource allocation mechanism is vital in regard to the objectives of both users and service providers because inefficient resource allocation mechanism in heterogeneous fog platforms burdens huge costs on providers' cash flow and the low quality of experience from the user's viewpoint. Consequently, the users eventually release unreliable providers [6]. In this regard, machine learning techniques such as the clustering approach can be beneficial to group available fog resources which can meet the requested resources submitted/asked by diverse IoT applications. In scenarios such as smart homes, multiple applications with variable resource requests are distributed simultaneously in cloud and fog nodes that provide different IoT-based services. The IoT aims to connect any device to the internet; for example, smart cameras, wearable gadgets, environmental sensors, home appliances, etc. [7]. Thus, it is generating a huge amount of data that can push the storage system and data analysis programs [8]. The IoT applications are proliferating on a large scale, so these data, services, and IoT applications are under pressure to move to the fog platforms due to their diversity and remoteness from cloud data centers. There are also many important safety programs such as health monitoring and emergency response systems that require the processing and transmission of IoT data with low latency [2, 9]. Delays due to data transfer to the cloud and then return to the application can seriously affect its performance and the overall QoS. To overcome this challenge, a fog-layered computing model has been proposed, where cloud-based services are provided to reduce network latency and congestion, as well as to support better mobility of users and location-aware devices & services; also, it expands the edge of the network due to the high geographical extent of the fog environment [10]. Fog Computing is a highly virtualized platform that provides computing, storage, and network services between end devices associated with users and cloud operating systems [11].

An important issue is the design of resource management techniques that determine, which analytical application modules are transferred to which computing node. If the technique and management are not used and the combined resources are provided to the programs, it will not be possible to cover the requirements of service quality and resource efficiency [12]. For this purpose, machine learning

techniques such as clustering and classification approaches have been used to provide resources to analytical programs requested by users efficiently [2, 13]. So, the aim of presenting this article is to improve delivered QoS to users by applying the k-means clustering approach for grouping available fog resources commensurate with resource requested applications in IoT and fog scenarios. As the canonical k-means clustering approach has an intrinsic high error rate [14], this paper proposes a genetic algorithm (GA)-based clustering optimization approach to optimize the allocation of resources to IoT applications in the layered fog computing environment. Since the majority of IoT applications are time-sensitive, the low delay and near physically allocated resources improve the amount of delivered QoS. Therefore, the resource clustering algorithms with the lowest distance error rate and the lowest delay as a consequence are favorable. The aim is to reduce clustering errors and improve the overall performance of the system. The approach involves clustering IoT devices based on their resource requirements and optimizing the allocation of resources to these clusters using a customized GA based on the stated problem. The GA is a searching algorithm inspired by the process of natural selection, and it is often used to solve optimization problems [15]. The algorithm starts with a random population of solutions, where each solution represents a possible clustering of devices. The fitness of each solution is evaluated using an objective function that measures the clustering error which is relevant to the distance between the device and fog node meaning the tolerable delay. The genetic operators of selection, crossover, and mutation are then applied to generate new solutions. The proposed approach has been evaluated using simulation experiments, and the results show the efficiency of the proposed algorithm against other state-of-the-arts. Therefore, the contribution of this article are as follows:

- We formulated the problem of resource allocation in fog platforms tailored with requested IoT applications to an optimization problem.
- We used a novel distance and error rate metric to measure the similarity of available resources to classify in the same cluster so that is the best suited for requested IoT applications.
- We applied the machine learning technique and the genetic algorithm for solving the stated optimization problem

To verify the proposed GA-based machine learning algorithm, several scenarios are conducted and the performance of the current proposal is compared against other strong counterparts. The results witnessed significant dominance in terms of objective metrics. The paper is organized as follows. The literature review including

background information and related works is placed in Sect. 2. The proposed system model is presented in Sect. 3. Problem statement and formulation are placed in Sect. 4. The proposed genetic algorithm is elaborated in Sect. 5. Simulation results are presented and analyzed in Sect. 6. Finally, Sect. 7 summarizes the paper and provides future directions for forthcoming works.

2 Literature review

In this section, related literature is presented. Firstly, it presents a brief review of background information in the first subsection; then, related works are brought in the forthcoming subsection to investigate the potential and research gaps.

2.1 Background information

This subsection introduces some of the basic concepts of computing, especially cloud computing, and IoT + Fog scenario, along with machine learning techniques that will be used as problem-solving tools.

2.1.1 Cloud computing

The biggest benefit of resource allocation in the cloud is that the user neither has to install software nor hardware to access the applications, develop the application, and host the application over the internet. In addition, there is not any limitation on the resource media and the exact places. The applications and data can be available from any place in the globe. Cloud providers can share their resources over the internet during resource scarcity [4, 16]. On the other hand, with the rapid development of network technology, cloud computing has been developed into many applications due to its powerful computing and storage capabilities. So, it has become a popular paradigm nowadays. Nevertheless, because the number of end-user devices is extremely increasing and also the real distance between the cloud service providers to end-users is becoming farther, the existing cloud models encounter several challenges such as network, cost, and delay parameters. For some time-sensitive applications, the cloud computing model is difficult to meet the requirement of quickly responding to the needs of terminal devices [17, 18].

2.1.2 IoT and fog computing

With the rapid advancement of information and communication technologies, there arises a critical issue that both the data rate and computation demand grow exponentially. For example, emerging 5G applications such as real-time

video streaming, augmented reality, interactive gaming, and self-driving, require advanced data communication, computation, and storage techniques to handle the complicated data processing and storage operations [19]. This poses a new challenge to the conventional cloud computing paradigm [20]. Fog computing is considered an extension of cloud computing to complement the shortcomings of cloud computing [21, 22]. In the fog environment, the storage and computational process are shifted to the edge network near devices to support time-sensitive applications and to decline the network pressure of cloud data centers [23, 24]. Therefore, fog resources are inherently heterogeneous and IoT requests are different, so unintelligent allocation of resources severely reduces the quality of services and the efficiency of infrastructure resources. To solve the problem, we use artificial intelligence and machine learning techniques in classifying and allocating resources tailored to requested resources for IoT applications.

2.1.3 Machine learning techniques

Machine learning is a branch of Artificial Intelligent (AI) including a series of approaches for data analysis and constructing data models for investigating and predicting autonomously. Machine learning has different tools and in particular, the clustering technique has been used due to adaptation to the problem. Clustering is a method for finding cluster structure in a data set that is characterized by the greatest similarity within the same cluster and the greatest dissimilarity between different clusters. For non-parametric procedures, the clustering algorithms are mostly based on objective(s) function including measures of the similarities or dissimilarities parameters. The clustering approaches are split into hierarchical and partition-based methods where the partition-based methods are the most utilized in different applications [25]. One of the most popular clustering versions is the k-means method which minimizes the sum squared distance error [26]. This similarity measure is most commonly and, in most applications, based on distance functions such as Euclidean distance, Manhattan distance, Minkowski distance, Cosine similarity, etc. For grouping objects in clusters, the Euclidean distance is used in this paper. In mathematics, the Euclidean distance between each pair of nodes in Euclidean space is the length of a direct line between the two nodes. This distance is measured via the Cartesian coordinates of the nodes that use the Pythagorean Theorem. So, it occasionally is called the Pythagorean distance. Take $X = \{I_1, I_2, \dots, I_n\}$ a set of n items. A clustering algorithm A_C splits items into k number of disjoint clusters C_1 through C_k so that the predetermined criteria are met. In this regard, take $C = \{O_1, O_2, \dots, O_k\}$ a set of centroids of

disjoint M clusters. Equation (1) calculates the total distance rate (TDR) of clustering algorithm A_C which splits items into k disjoint clusters.

$$TDR(A_C(X, k)) = \sqrt{\sum_{i=1}^n \sum_{j=1}^k y_{ij} \times (I_i - O_j)^2} \quad (1)$$

The decision binary variable y_{ij} indicates whether I_i belongs to cluster C_j or not. Note that each item belongs only to one cluster; so, Eq. (2) stipulates this point.

$$\sum_{i=1}^n \sum_{j=1}^k y_{ij} = 1 \quad (2)$$

As Eq. (1) shows the total distance rate of each clustering algorithm A_C which clusters data of super set X to different clusters, this function indicates how efficient the clustering technique is good. In other words, the lesser gained value in Eq. (1) is considered as a lesser error rate. Therefore, it can be taken as an optimization algorithm. Since the majority of IoT applications are time-sensitive, the low delay and near physically allocated resources improve the amount of delivered QoS. Therefore, the resource clustering algorithms with the lowest total distance rate and the lowest delay as a consequence are favorable.

2.2 Related works

This subsection surveys related works; then, it presents a summary table which investigates state-of-the-arts with related metrics. Singh et al. [27] have presented resource allocation procedures for high performance computing (HPC) data centers that maximize value and minimize energy consumption. The methods compound the recognition of efficient allocation and convenient voltage/frequency levels to optimize value and energy consumption for tasks containing related assignments. This procedure considerably reduces energy consumption while ameliorating value. Gao et al. [28] presented a model to maximize resource utilization and reduce bandwidth costs in the cloud computing environment. They have introduced a hierarchical multi-agent optimization (HMAO) algorithm, which combines two algorithms, a Genetic Algorithm (GA) and a multi-factor optimization algorithm (MAO). Azimi et al. [29] proposed a swarm-based intelligence algorithm that utilizes multi-cluster techniques to solve resource management in the Fog environment. Devarasetty and Reddy [30] have proposed an optimal resource allocation method by considering the deployment cost and QoS requirements as the major objectives. To find the best discrete solution for the problem, the genetic algorithm has been used. With this solution, selecting the optimal

resource as a big task for the customers within the complex pricing structure is solved, then the customers can choose the optimal resources for their application deployment, through the cloud service providers. Wu et al. [31] proposed a resource allocation model to optimize both agreement index and robustness in IoT applications. To this end, a fuzzy logical offloading strategy has been proposed. To learn and optimize the fuzzy offloading strategy from different applications, a multi-objective Estimation of Distribution Algorithm (EDA) is designed. To save the system resources in a reduced search space by making scheduling decisions, The algorithm divides applications into autonomous clusters and for further processing, each cluster can be allocated to the corresponding tier. Li et al. [32] have proposed an energy-efficient computation offloading and resource allocation (ECORA) scheme to solve and surveyed the computation offloading and resource allocation problem of cloud and fog computing in the IoT context. This problem is formulated to minimize the system cost with a constrained optimization model. Jia et al. [33] have proposed a strategy by surveying the computing resource allocation problem in three-layer fog computing networks. For this proposal, first, a double two-sided matching optimization problem, from the resource allocation problem is formulated. Afterward, based on cost efficiency, a double-matching strategy for the resource allocation problem in fog computing networks have proposed, by analyzing the utility and cost in fog computing networks is derived. Adopting this proposed strategy brings high-cost efficiency performance. Zhou et al. [20] have developed a pricing-based stable matching algorithm to address the task assignment problem, and to motivate vehicles to share their resources have proposed. The suggested algorithm was a contract-based incentive mechanism by investigating the computation resource allocation and task assignment problem in Vehicular fog computing (VFC) from a contract-matching integration perspective. Mebrek et al. [34] have considered energy consumption and quality of service as two essential criteria in fog processing performance and introduced workload allocation using a Genetic algorithm and the broadcast incremental power (BIP) as a solution. For this solution, the fog architecture is modeled as a unidirectional graph. But, it has a lack of change in energy consumption and delays in the low and medium scale of the Internet of Things, which is the disadvantage of this approach. Abbasi et al. [13] have presented a new learning classifier system (LCS) called Best Classifier Memory- extended classifier system (BCM-XCS) as the most important innovation. For this innovation, a method for optimizing the partial distribution of workloads and creating a balance between delays and power consumption at the edge was proposed. For the first time by adapting the parameters including total workload

and the amount of green energy as the input of the system, XCS tries to find optimal classifiers by interacting with the environment and receiving rewards as well as using the genetic algorithm. Moreover, an additional memory was utilized in the XCS system to accelerate the learning process and the optimal form of workload distribution was computed. By storing the sequence of the input conditions of the system along with the best action for each state in the secondary memory, in this new XCS-based classifier, the application of obscure random actions is reduced. A considerable increase in the accuracy of the LCS is one of the major results of this new operation. In another work, Abbasi et al. [35] have proposed a processing model for the problem of allocation of workloads in IoT-fog-cloud architecture in which a trade-off between the energy consumption of fog and propagation delay of cloud in processing workloads by fog and cloud is formulated. They have solved this multi-objective model of the problem by using non-dominated sorting genetic algorithm II (NSGAI) algorithm. It satisfied both of energy-consumption and delay criteria. Pourkiani and Abedi [36] have addressed the task distribution in combined fog-cloud scenarios in the online healthcare applications and presented the Machine Learning Based Task Distribution (MLTD) technique. For the training process of the artificial neural network (ANNs), utilized Levenberg–Marquardt (LM), and Genetic Algorithm (GA) have been used. As an advantage, it can be said that the MLTD technique is scalable and the proposed method works perfectly for the task distribution process when the servers provide different response times, but it is not suitable for situations in which servers provide almost similar response times. It can conclude that the performance of MLTD entirely depends on the error of the ANNs for the predictions model. Table 1 is dedicated to summarize the related works based on prominent metrics.

None of the methods discussed above have comprehensively addressed resource allocation for fog nodes by clustering. Therefore, many researchers are now attempting to study this issue. The present study follows the same research trend to minimize the overall delay and deliver efficient resource management at the same time.

3 System model

Cisco's three-layer network design model is a very broad design in the information technology (IT) industry to build a credible, scalable, and cost-effective Internet business. This design includes three layers: access, distribution, and core [37]. Figure 1 illustrates a typical 3-layered hierarchical IoT-Fog-Cloud system model.

The architecture depicted in Fig. 1 is a general model and not customized for different applications. Although the Cisco three-layer model covers the weaknesses of the cloud data centers, we can cluster resources in different layers with more layering in the fog environment to achieve higher efficiency. The proposed system model is well-suited for health-care and smart homes applications illustrated in Fig. 2.

This section presented the implementation of a 4-layer fog computing architecture for the smart home and discussed its design details that it is for better demonstrating the use of hierarchical fog architecture concerning optimal resource allocation. It is followed by a layered resource model and application model. The layers are as follows:

IoT device layer (L_0): IoT devices are the end devices as sensors that are placed at the edge of the network.

Countless sensors are in these devices, which are very reliable and inexpensive. So, they can distribute in public infrastructure extensively and with this distribution of end-devices can service multiple applications. It should be considered that these spatially distributed end-device sensors generated a huge amount of data which must be processed coherently and in a general manner.

Fog Layer 1 (L_1): It consists of many low-power, low-latency computing nodes or edge devices, close to data sources and users. So, the end devices transfer raw data to this layer. For being responsible for covering a neighborhood or a small community, and analyzing data promptly, each edge device in Layer 1 connects to a group of local IoT devices. The output of these edge devices at this level has two parts: (1) send the results of processing heavy work to an intermediate computing node in its next upper layer, (2) to respond to separate computations, and lightweight calculations of infrastructure components, sent simple, short, and fast feedback to the local infrastructure. In addition, because of the lack of trust in their limited storage resources, the nodes in this layer do the data transfer. These nodes have low storage, medium computing resources, and network resources. However, compared to higher layers, they have very low access delays.

Fog Layer 2 (L_2): Nodes in this layer contain medium resource settings and they can handle medium-scale data processing and simple calculations, but for long periods cannot be used to store data, and each of them is connected to a group of fog nodes in Layer 1.

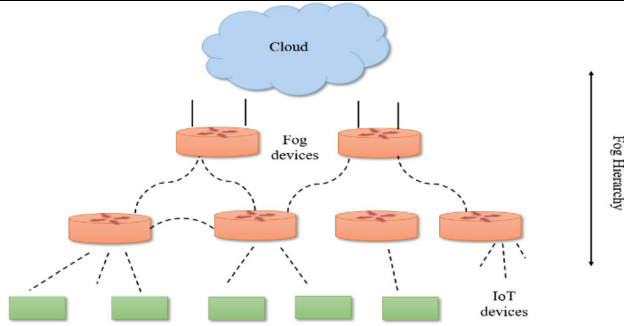
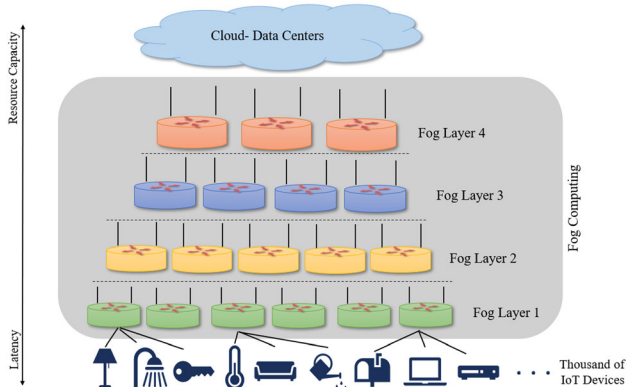
Fog Layer 3 (L_3): These nodes can be used for basic analysis because several intermediate computing nodes have high resource settings. Due to the lack of historical data and limited resources, data cannot be stored in these nodes for a long time and large-scale analysis is not possible.

Table 1 Summary of related works and their comparison based on prominent metrics

Author(s)/ (Ref.)	Applied tools	Category	Objective(s) to optimize	Description
Singh et al. [27]	GA algorithm	Meta-heuristic-based	Multi-objectives: maximize value, minimize energy consumption	Dynamic voltage and frequency scaling (DVFS)-aware profiling and non-profiling-based methods were presented, which employ design-time profiling outcomes and do all computations at run-time, respectively
Gao et al. [28]	HMAO: GA and MAO algorithm	Meta-heuristic-based	Multi-objectives: resource utilization, and bandwidth usage	Have aimed to maximize the resource utilization based on CPU, memory, and GPU, and minimize the bandwidth cost
Devarasetty and Reddy [30]	Genetic algorithm	Meta-heuristic-based	Multi-objectives: QoS performance, deployment cost	For resource allocation that considers the objectives of minimizing the deployment cost and improving the QoS performance, have proposed the improved optimization algorithm
Wu et al. [31]	Fuzzy clustering method, EDA and HEFT, DFHEFT, MOEA/D algorithm	Meta-heuristic-based	Multi-objectives: agreement index and robustness	To solve the uncertain computation offloading problem in IoT–fog–cloud systems, an evolutionary algorithm-guided fuzzy rule partition method has been proposed. On two agreement index and robustness objectives algorithm has performed well
Li et al. [32]	ECORA: convex optimization tools CVX, TCCO scheme	Meta-heuristic-based	Multi-objectives: energy-efficient computation offloading and resource allocation, minimize the system cost	The computation offloading problem of the coexistence and synergy between fog computing and cloud computing in IoE by jointly optimizing the offloading decisions, the allocation of computation resources, and transmit power has been investigated
Jia et al. [33]	DA-DMS: DA algorithm	Meta-heuristic-based	Single-objective: cost efficiency	Has been developed a double-matching strategy based on the deferred acceptance algorithm (DA-DMS) by investigating the resource allocation problem in three layers of the fog computing networks, and by adopting the DADMS strategy, high-cost efficiency performance could be achieved
Zhou et al. [20]	contract theory and Matching algorithm	heuristic-based	Single-objective: Task assignment	Based on contract theoretical modeling, an efficient incentive mechanism has been proposed. Then, has been transformed the task assignment problem into a two-sided matching problem between vehicles and user equipment (UEs). Finally, the formulated problem was solved by a pricing-based stable matching algorithm
Mebrek et al. [34]	Improved Genetic algorithm and BIP algorithm	Meta-heuristic-based	Multi-objectives: energy-consumption and delay	For investigating the power consumption and delay in fog-cloud computing and traditional cloud computing, has been presented a model
Abbasi et al. [13]	BCM-XCS: Genetic algorithm	Meta-heuristic-based	Multi-objectives: partial distribution of workloads power consumption and delay	Has been proposed two methods based on XCS learning classifier systems (LCS), namely, XCS and BCM-XCS, to balance the power consumption at the edge of the network and to reduce delays in the processing of workloads
Abbasi et al. [35]	NSGAIL: Genetic algorithm	Meta-heuristic-based	Multi-objectives: energy-consumption and delay	Has been presented a processing model, which can formulate a trade-off between energy consumption and delay in processing workloads in fog

Table 1 (continued)

Author(s)/ (Ref.)	Applied tools	Category	Objective(s) to optimize	Description
Pourkiani and Abedi [36]	MLTD: Levenberg– Marquardt (LM) and Genetic Algorithm	Meta- heuristic- based	Multi-objectives: efficient utilization of Internet bandwidth and reducing the response time resource utilization	For using the Artificial Neural Networks for smart task distribution between the fog and cloud servers, Machine Learning Based Task Distribution (MLTD) technique has been proposed. Then, to detect the parameters that can impact its performance, It evaluates the efficiency of MLTD in different conditions

**Fig. 1** A typical 3-layer hierarchical architecture [37]**Fig. 2** A presented customized hierarchical fog architecture for smart home

Fog Layer 4 (L_4): These fog nodes are used for supporting local analysis and storing data for short periods. Note that these nodes associated with layer 4 have sufficient resource capacity.

Cloud Layer (L_5): It is a metaphor for a cloud computing data center at the top layer and it has sufficient resources. This layer, in the lowest layers from each intermediate computing node, collects data and information. At this layer, because of distributed computing performance and cloud storage capacity, it can perform complex, long-term behavior analysis to support dynamic decision-making, such as long-term pattern identification, large-scale event detection, and relationship modeling. In comparison with fog nodes in the lowest layers, servers

in the cloud layer have more latency because of far distance [38].

3.1 Resource and application model

The considered layered resource model is a heterogeneous fog resource model including n number of available heterogeneous computing resources $R = \{r_1, r_2, \dots, r_n\}$ where each item is a vector of computing resources (processor, memory, and bandwidth) so that each resource r_j belongs to only of the layers L_1 through L_5 . In that available time interval, the set of m resource requests of IoT applications are received known as application request (AR) set $AR = \{(Req_1, Tdel_1), (Req_2, Tdel_2), \dots, (Req_m, Tdel_m)\}$ where the terms vector Req_i and time $Tdel_i$ are requested computing resources for IoT applications A_i and their tolerable delay. The maximum delay can be measured based on the distance between the device and allocated node which must be less or equal in comparison with the $Tdel_i$ parameter. In the proposed architecture, we assume that according to the fog nodes' resources, they have been layered and each application that has a resource request is sent to the fog layer, which can meet the requirements of that application. In short, the four-layer fog computing has high computational performance and this architecture supports fast response at various levels, so it is efficient for optimal resource allocation in future smart homes. The main goal is to group the layered resources into clusters; then, the resources are allocated to the IoT applications (from devices) which issued the resource request because the resource is well-suited for meeting their desired QoS. In other words, the requester devices are joined to the most appropriate cluster of fog resources.

4 Problem statement

This section explains, after the desired layer has been determined, how to find the appropriate fog nodes for resource provisioning to given IoT devices by clustering

Table 2 Used nomenclature

Symbol	Description
R	Set of heterogeneous fog nodes
n	Number of available fog nodes
r_j	Resource vector of j -th fog node
AR	Set of heterogeneous requested resources
Req_k	Requested resource vector of k -th IoT application
x_j	j -th IoT device
$x_{j,i}$	i -the resource dimension of j -th device
d	Number of resource dimensions
m	Number of requested IoT devices (for their application)
C	Number of clusters each of which contains number of fog nodes
K	Number of all resource clusters
C_j	j -th fog cluster
l_j	Number of elements in j -th cluster
y_{ij}	Binary decision variable determines.
O_j	The center fog node of j -th cluster
x	A device x mapped to cluster C_j

method. Table 2 is dedicated to the nomenclature used in the problem statement section. It makes easy to follow.

There is a set $R = \{r_1, r_2, \dots, r_n\}$ of n number of heterogeneous fog nodes. The IoT applications submit their requests; it is received known as application request (AR) set $AR = \{(Req_1, Tdel_1), (Req_2, Tdel_2), \dots, (Req_m, Tdel_m)\}$ which meets users' desired QoS. The aim is to allocate requested resources to IoT applications (issued by devices) so that the total error rate is minimized; this rate indirectly burdens service delivery delay on desired QoS by incorporating a fusion of machine learning and meta-heuristic genetic algorithm. In the used clustering technique, an important step is to select a distance measure, which will define the similarity between two elements. For this problem, the distance between each IoT device and all fog nodes must be measured which indirectly determines the service delay. This similarity is measured by the Euclidean distance, Eq. (3) is used by the Euclidean distance.

Euclidean distance can be generalized to determine the distance between two d -dimensional vectors $A = (A_1, A_2, \dots, A_d)$ and $B = (B_1, B_2, \dots, B_d)$.

$$dist(A, B) = \sqrt{\sum_{j=1}^d (A_j - B_j)^2} = \|A - B\|_2 \quad (3)$$

Firstly, available fog nodes are grouped in several clusters of fog nodes. To this end, a Jaccard similarity is applied. For instance, the Jaccard similarity (JS) of two clusters $C_j = \{I_1, I_2, \dots, I_p\}$ and $C_l = \{J_1, J_2, \dots, J_q\}$ is calculated by Eq. (4).

$$JS(C_j, C_l) = \frac{1}{p \times q} \sum_{j=1}^{|C_j|} \sum_{l=1}^{|C_l|} dist(I_j, J_l) \quad (4)$$

By the introduction of Jaccard similarity, the pairwise comparison in heterogeneous fog nodes is done hierarchically till all of the K clusters are formed; namely, $C = \{C_1, C_2, \dots, C_K\}$. The aim is that the sum of the Jaccard similarity of all pairs is minimized. In other words, elements in a cluster are near to the cluster center and are far from another center. Secondly, the m number of IoT devices which run their applications are joined to different clusters in the same manner; so, the resource of that cluster is allocated to the IoT device to satisfy its requirement. Equation (5) measures the distance between a device x_j which requests Req_j to the cluster C_k having O_k as its center.

$$dist(x_j, C_k) = \sqrt{\sum_{i=1}^d (x_{j,i} - O_k)^2} \quad (5)$$

This paper aims to reduce the latency and to provide optimal fog nodes resource allocation to the IoT devices by reducing the within clustering distance (WCD) metric. Therefore, to have the most similarity between the members of the cluster and the least similarity between the clusters. If devices are joined to different clusters, Eq. (6) measures the WCD metric.

$$WCD = \sum_{k=1}^K \sum_{x \in C_j} dist(x, O_k) \quad (6)$$

Here, the term K is the number of total fog clusters. The WCD is within cluster distance and also is considered an objective function that must be minimized. Therefore, it can be considered the sum of the minimum distances x from the data centers (m). So, the problem is how to join devices to the cluster of fog nodes where the within clustering distance is minimized. In other words, the resources in the fog clusters are allocated to devices so that the WCD metric is minimized. Therefore, the problem is formally stated as optimizations, and the objective function is drawn in Eq. (7).

$$Fitness = \min \sum_{i=1}^m \min_{1 \leq j \leq K} dist(x, C_j) \quad (7)$$

Therefore, this is an NP-hard computational problem [13, 32, 39], so, an evolutionary intelligent algorithm is required to solve it.

5 Proposed customized genetic algorithm for solving the stated problem

As mentioned in the problem statement, the explained problem is an NP-hard problem and an evolutionary algorithm is required to solve it. Due to various experiments and due to the flexibility and high efficiency of genetic algorithm, a customized genetic algorithm is used. The elementary items of the customized GA that are how to encode real world problems, fitness function, termination criteria, etc. are explained.

5.1 Problem encoding (gene and chromosome)

To generate the initial population in the proposed method, datasets are first created that are coordinate locations for all fog nodes and requests of all applications in the search space (for smart home as a case study). These values are generated randomly with a uniform distribution in a certain interval. In genetic algorithms, data are first presented at the stage of production of the initial population, which is considered the answer to the problem. Each answer is called a chromosome similar to Fig. 3, and any information that contains in a chromosome (for example, the value of x coordinates and the value of y coordinates) is called a gene. Then, using genetic algorithm operators, after selecting better chromosomes, crossover and mutation are performed on the chromosomes. Eventually, the current population merges with the new population resulting from crossover and mutations in chromosomes.

5.2 Fitness function

As mentioned in the previous sections, the purpose of this paper is to optimally allocate the resources of fog nodes to IoT applications from their devices. The clustering technique has been used for optimal resource allocation. Most of the objective functions used in the clustering problem typically work based on both compaction and separation indices. The compaction criterion refers to the scatter between data within a cluster or between data and clusters header that should be minimized. To better optimize the clustering and allocation of nodes to the requesting IoT devices, using a genetic algorithm, an attempt has been made to minimize the distance within the cluster, which is the reduction of the error rate in the k-means technique. Here, the objective function is to minimize cluster error or WCD value which is corresponding to the fitness function of each solution. As mentioned earlier, the minimum error percentage (Error squared aggregation) is calculated by Eq. (4).

5.3 Exploration and exploitation

In meta-heuristic algorithms, two important operations are exploration and exploitation, known crossover as exploration and mutation as exploitation in the genetic algorithm. Crossover is the most important step in a genetic algorithm and is a process for producing new chromosomes. A crossover is made between two chromosomes that can produce two new chromosomes. Gene traits are shared at the crossovers. In the genetic algorithm, several crossover operators are introduced, in this paper, a uniform crossover is used.

Uniform crossover This crossover exchanges genes from one chromosome to another based on probability. Each gene is like a coin, for example, if it appears on the coin, it is exchanged, and if it appears under the coin (lion), the position of the gene remains. Figure 3 shows the utilized uniform crossover.

In a uniform crossover, as can be seen, an α is formed, which is a random probability, and as a result of this probability, the parents are combined. The next step is to mutate the new children. Genetic algorithms have a small and constant probability of change. Based on this probability, the child's chromosomes change or mutate

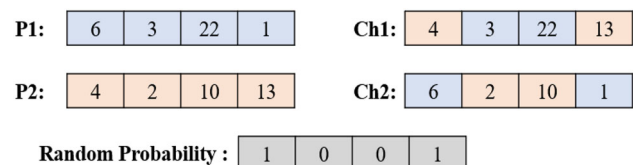


Fig. 3 Discrete uniform crossover

P1:	6	3	22	1
------------	---	---	----	---

Random mutation Probability :	6	1	22	1
--------------------------------------	---	---	----	---

Fig. 4 Discrete mutation

randomly; with mutating chromosome genes in the data structure are changed. Figure 4 shows how the mutation operator changes the given solution.

As shown in Fig. 4, the mutation operator in discrete problems is as follows. Firstly, with the probability that the number of genes for the mutation is determined depending on the defined problem, then the second gene replaces the first gene. There are different mechanisms for selecting chromosomes for crossover and mutation (such as the roulette cycle, tournament, etc.), in this article, the roulette cycle selection mechanism is used. The selection mechanism, known as the roulette wheel, is the most popular and

widely used method. The logic of the roulette wheel mechanism says, in the roulette wheel mechanism, each of the chromosomes is likely to be selected depending on the degree of suitability or fitness value (based on the error function). In other words, the better a chromosome is, the more likely it is to be selected to produce the next generation, and conversely the worse the chromosome, the less likely it is to be selected to produce the next generation.

5.4 Termination criteria

There are several procedures for the termination of the algorithm. For this problem, the number of repetitions procedure is used. It means, when the number of iterations of the main loop of the algorithm reaches a certain value, the termination condition is realized. Figure 5 and Algorithm 1 are the block diagram and presented pseudo code of the proposed improved genetic algorithm respectively.

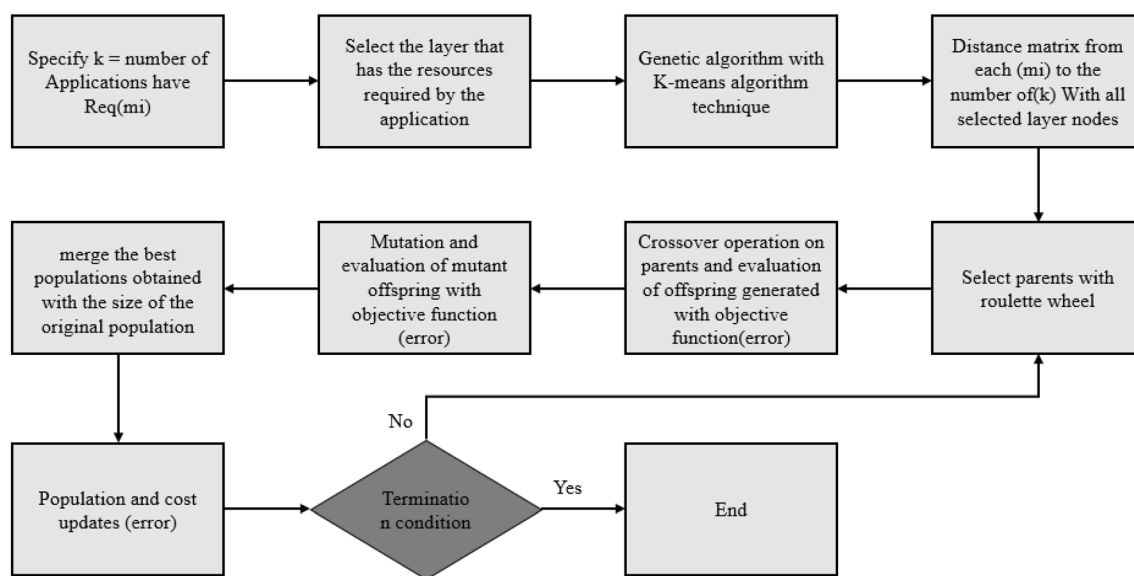


Fig. 5 Block diagram of the proposed algorithm

Algorithm 1: GA for resource allocation**initial**

```

1:  $l \leftarrow$  define as a struct, including nodes of  $L1, L2, L3, L4$ 
2:  $j \leftarrow L1$  or  $L2$  or  $L3$  or  $L4$ 
3:  $nl \leftarrow$  number of layers
4:  $ReqD \leftarrow$  Resource of requested IoT devices
5:  $sum\ l.j \leftarrow \text{sum}(l.j\{cpu, Mem, Bandwidth\})$ 
6:  $sum\ R \leftarrow \text{sum}(ReqD\{cpu\ Mem, Bandwidth\})$ 
7: Find in  $nl$  layers by comparing  $sum\ l.j$  with  $sum\ R$ 
8: If  $sum\ R > sum\ l.j$ 
9:   Reject  $ReqD$ 
10:  Return failed
11: Else
12:   Accept  $ReqD$ 
13: End
14:  $X \leftarrow (l.jx, y)$ 
15:  $xy \leftarrow (ReqDx, y)$ 
16:  $k \leftarrow \text{number of}(ReqD)$ 

```

GA Parameters

```

17:  $MaxIt \leftarrow$  End condition (maximum number of Iteration)
18:  $nPop \leftarrow k$  (Population size)
19: For each  $Pop : nPop$ 
20:    $Pop.position \leftarrow xy$ 
21:    $Pop.cost \leftarrow$  Cost Function ( $Pop.position$ ) by calling
   Algorithm 2
22: End
23: Sort  $Pop$  based on costs
24: Store Best solution and Worst solution

```

Main Loop

```

25: For each Iteration:  $MaxIt$ 
26:   Find Parents with Roulette Wheel Selection
27:   Crossover operation
28:   Evaluate offspring with Cost Function ( $Popc.position$ )
   by calling Algorithm 2
29:   Select a random Parent
30:   Mutation operation
31:   Evaluate mutant with Cost Function ( $Popmu.position$ )
   by Algorithm 2
32:    $Pop \leftarrow$  Merged population of ( $Pop$ ), ( $Popc$ ), ( $Popmu$ )
33:   Sort  $Pop$  based on costs
34:   Truncation  $Pop$  to  $nPop$ 
35:   Update and Store Best solution and Worst solution
36: End

```

In algorithm 1, the pseudo-code of the optimal resource allocation program is presented. Firstly, the fog nodes in the hierarchical layers and resource requesting programs

are received from the input (Algorithm 1, lines 1 to 4), then the sum of the processors of each layer, with the sum of processors required by the device are performed. The

requesting layers are compared to determine the server layer, if the amount of requested processor resources is more than the 4-layer fog resources, it must be sent to the cloud environment, so it is not acceptable in the fog environment (line algorithm 5 to 13).

The coordinates information of the fog nodes of the selected layer are in X and the coordinates information of the applications are in xy , and the number of requesting applications is in k at this specified time (Algorithm 1, lines 14 to 16).

The next section introduces the parameters of the genetic algorithm, the number of iterations that are considered as the termination condition is in MaxIt , nPop here contains the population size value, which is the same as k (Algorithm 1 lines 17 and 18). The coordinates of each population sample are stored in Pop.position and then its cost is calculated (Algorithm 1 lines 19 to 22). Populations are sorted in ascending order based on cost function values, and the best and worst solutions are stored (Algorithm 1, lines 23 and 24). In the main loop of the program, in each iteration using the roulette wheel, the parents are selected for the crossover operator and the crossover operation is performed and the generated children are evaluated with the objective function (Algorithm 1 lines 25 to 28), then randomly a parent is selected for the mutation operation. The mutation operation is performed on it; then, the results is evaluated by the cost function (Algorithm 1 lines 29 to 31); note that, the objective function is calculated by Algorithm 2. Now, the initial population and the offspring created by the crossover and the mutation are merged; then, the population must be arranged in ascending order based on the cost function. After that, we consider the best population at the top, in the number of nPop , as the main population, and finally, the best and worst solutions are updated and stored until the loop ends (stop condition is met that can be looked in Algorithm 1 lines 32 to 36).

stores it in $dmin$; then, the WCD , which is the sum of the stored $dmin$ has been obtained and returns as an optimal output (Algorithm 2 lines 1 to 4).

6 Simulation and evaluation

In optimization algorithms that are used to optimize clustering, a set of solutions with different numbers of clusters is presented instead of a single solution. Therefore, in the proposed method, the k-means algorithm model is used. First, all fog nodes are considered as a cluster, then, the best, i.e., the least delayed solutions are selected by Euclidean criterion.

In this section, a random data set is used to evaluate the proposed method. These collections include the fog nodes section and IoT applications that request resources. Then, the performance of the proposed GA algorithm is compared with the customized particle swarm optimization (PSO) [40], customized differential evolution algorithm (DE) [41], customized Harmony Search (HS) [42], and customized Artificial bee colony (ABC) [43] algorithms. After introducing and explaining this data set, the results of the experiment will be reviewed. We examine the prototype of a smart home. There are many types of IoT data that can be collected and analyzed in a smart home. Some of the most common types of IoT data for smart homes are demonstrated in Fig. 6.

The types of IoT data collected in a smart home will depend on the specific devices and systems installed, as well as the preferences and requirement of the home's occupants. We assume that there are a total of 375 fog nodes in an area of 50×50 square units with 200 nodes in the first fog layer, 100 nodes in the second fog layer, 50 nodes in the third fog layer, and 25 nodes in the fourth fog layer. The location of each fog node was randomly gen-

Algorithm 2: Objective Function

Cost Function (X, xy)

- 1: $d \leftarrow$ calculate the distance matrix between each X, xy
 - 2: $dmin \leftarrow$ **Find** the closest distance by minimum d
 - 3: $WCD \leftarrow$ sum ($dmin$) based on Eq. (6)
 - 4: **Return** WCD
-

In algorithm 2, the quasi-code is the objective function, the objective is the cost reduction function whose input parameter is the coordinates of the fog nodes and the coordinates of the IoT applications that request the resources. It firstly calculates the distance between all nodes and applications in the distance matrix (d), then obtains the closest distance through the minimum (d) and

erated by a uniform distribution in the coordinate space (0,0) ... (99,99). It is assumed that 4 applications with different requests, which are also in the coordinates (0,0) ... (99,99) need IoT resources. Resource values include non-negative integers distributed with a uniform distribution. Table 3 provides a brief description of the data set, and this data set is layered according to the hierarchical

architecture model presented in the system model of Sect. 4. The fog nodes processing capacity is determined by Million Instruction per Second (MIPS) metric.

In this section, we consider the CPU capacity as the main source. Unlike traditional cloud servers, fog nodes can be detected through their physical locations. In our model, this physical position helps to find the nearest fog node in the clustering algorithm.

Applicant Device Model We use the concept of task application for computational tasks that must be performed using cloud or fog sources. The request is modeled in Eq. (6).

$$\text{ReqD} = \bigcup_{i=1}^k \text{reqd}_i \quad (6)$$

Equation (6) is a set of task applications (resource-requesting devices) that are placed in a fog environment. A request is described as ReqD and it is specified by the IoT issuing the application. Table 4 brings the information associated with the resource request issued by each application.

Table 4 lists the requested devices and their required resources that were examined in this study. To test this proposed algorithm under different conditions, we randomly generated four applications, each of which with a different number of requests, with a uniform distribution, and tested them with different meta-heuristic algorithms. Also, according to the sources on demand and their position, they receive services from different layers in which they are located. For example, the first application receives resources from the first layer according to the location and resources requested, and the other three applications receive resources from the second layer because the total of 80 requested resources (CPU) is more than the total resources (CPU) of the first layer. In terms of location, it is

Table 3 Dataset information of fog nodes

Dataset	CPU (MIPS)	x	y	Number
Layer1	10–50	0–40	0–40	200
Layer2	51–80	41–69	41–69	100
Layer3	81–110	70–89	70–89	50
Layer4	111–130	90–99	90–99	25

located near the second layer. In the following, the result obtained by comparing these applications with the mentioned algorithms is shown. All algorithms have been run 40 times independently. The statistical results including the min, max, and average values are reported. Table 5 puts the information about statistics of the executions.

In this regard, Figs. 7, 8, 9 and 10 are dedicated to visually presenting the comparison of comparative algorithms based on newly defined optimization function.

To evaluate the proposed algorithm, we compared it with other meta-heuristic algorithms, and to evaluate it under different conditions, we examined it 40 times for all four Applications. 5 and the four Figs. 5, 6, 7, 8, 9 and 10 show the details regarding the statistical evaluation of each program's implementation across 40 rounds from 4 scenarios. As the figures demonstrate, the proposed GA has the most optimization level (error reduction) among the five algorithms. It had better *min* and *Max* values of the optimization (Min Opt, Max Opt) in the four scenarios. It also has already the best amount of average optimization value (Avg Opt). The PSO is the second finest, it just had the greater value in the Max Opt of the second scenario which was requested the resources of layer 2. This was then followed by the value of the error reduction of DE. The HS and ABC have the least amount. In the first two scenarios, ABC has outperformed HS in regard to results.

Fig. 6 Schematic of data type and transfer of smart homes to fog nodes

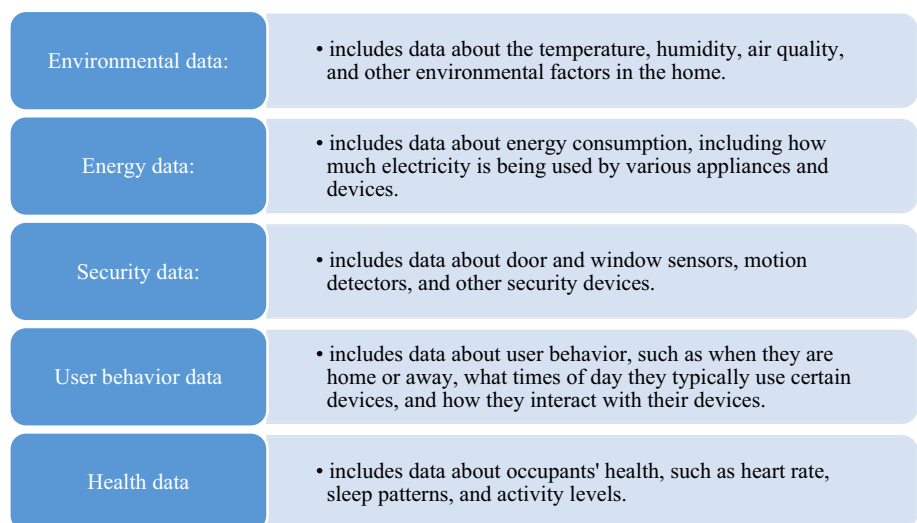
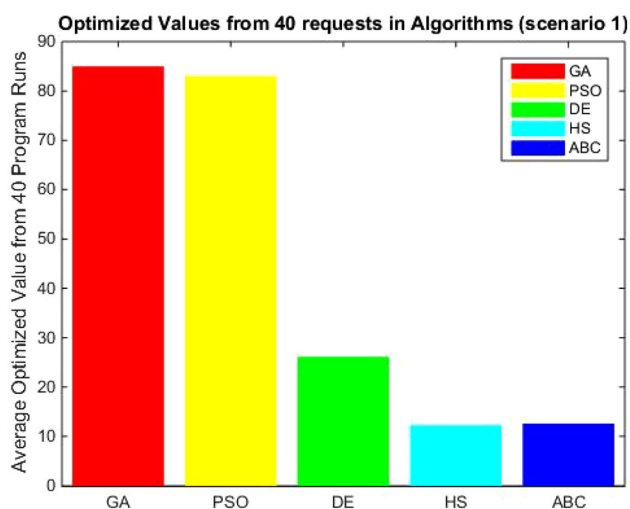


Table 4 IoT application with required resources

Number of scenarios	Services	Num of ReqD	CPU(MIPS)	x	y
1	App1	40	12–133	0–40	0–40
2	App2	80	12–140	41–69	41–69
3	App3	150	10–30	0–40	0–40
4	App4	200	5–25	0–40	0–40

Table 5 Statistical analysis of the implementation of 40 rounds of each program

Statistical analysis	Min opt	Max opt	Avg opt	Min time	Max time	Avg time
App. with 40 Req						
GA	71.49	92.82	84.83	18.18	19.46	18.67
PSO	63.68	92.30	82.92	17.29	18.45	17.71
DE	17.87	34.23	26.01	20.39	22.58	20.79
HS	2.43	19.32	12.13	67.47	69.77	68.31
ABC	4.49	20.94	12.45	30.87	42.90	32.05
App. with 80 Req						
GA	54.67	63.79	60.34	34.87	37.90	35.93
PSO	46.33	66.38	58.49	34.98	67.32	38.95
DE	10.17	19.51	15.59	20.88	27.76	22.64
HS	1.88	8.01	5.70	94.78	182.12	101.53
ABC	3.57	11.84	6.98	63.80	65.27	64.48
App. with 150 Req						
GA	86.42	103.00	93.57	108.02	114.96	109.44
PSO	39.67	99.15	79.91	103.88	135.92	117.83
DE	9.29	24.41	17.06	33.48	44.23	34.28
HS	2.09	12.70	8.61	166.42	412.23	184.07
ABC	0.00	9.56	4.52	195.45	200.71	197.50
App. with 200 Req						
GA	85.90	97.68	92.59	163.10	168.91	164.91
PSO	46.97	92.85	73.18	175.98	225.50	187.67
DE	10.85	21.79	15.84	38.19	41.90	38.83
HS	0.00	14.30	7.32	231.98	306.98	249.64
ABC	0.00	8.17	3.70	291.97	299.61	294.43

**Fig. 7** Algorithms comparison in terms of evaluating the amount of Optimization from the 40 rounds of implementation of Scenario 1

The first scenario:

we considered the application with 40 requests in the smart home environment, which is located in the first layer of fog. Figure 7 shows the comparison results of GA algorithms for this scenario. The average amount of error reduction associated with the WCD metric with the genetic algorithm is equal to 84.83, this value with the PSO algorithm equal to 82.92, with the DE algorithm equal to 26.01, with the HS algorithm is equal to 12.13 and with the ABC algorithm is equal to 12.45. As can be seen, our proposed genetic algorithm has had better results than other algorithms when the number of requests is low.

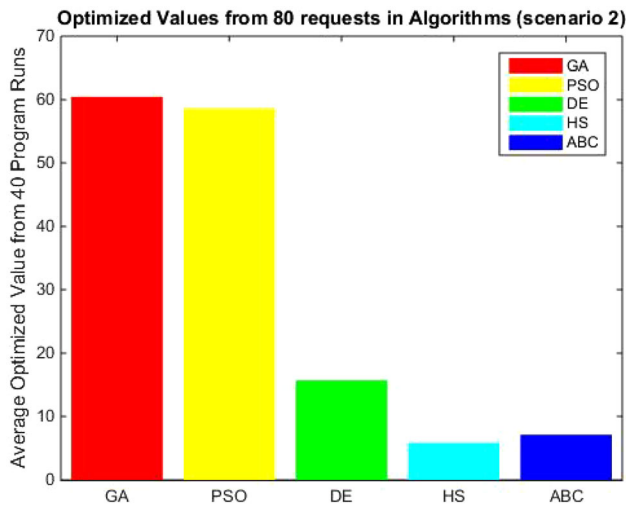


Fig. 8 Algorithms comparison in terms of evaluating the amount of Optimization from the 40 rounds of implementation of Scenario 2

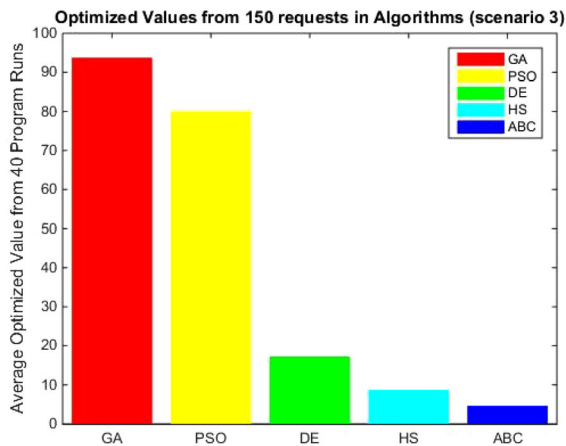


Fig. 9 Algorithms comparison in terms of evaluating the amount of Optimization from the 40 rounds of implementation of Scenario 3

The second scenario: we considered the application with 80 requests in the smart home environment, which is in the second layer of Fog. Figure 8 shows the comparison results of GA algorithms for this scenario. The average amount of error reduction associated with the WCD metric with the genetic algorithm is equal to 60.34, this value with the PSO algorithm is equal to 58.49, with the DE algorithm is equal to 15.59, with the HS algorithm is equal to 5.7 and with the ABC algorithm is equal to 6.98. As shown in Table 5, in the second program, due to the high range of CPU resource requests and for physical longer distances, the error reduction rate is lower in all algorithms, however, our proposed genetic algorithm has had better results.

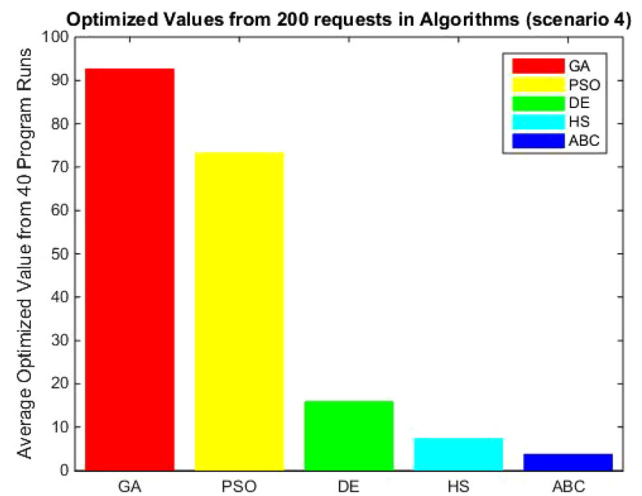


Fig. 10 Algorithms comparison in terms of evaluating the amount of Optimization from the 40 rounds of implementation of Scenario

The third scenario:

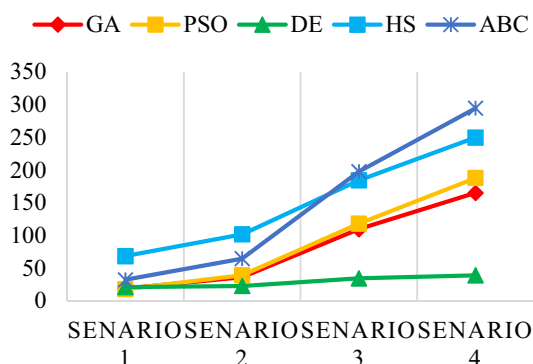
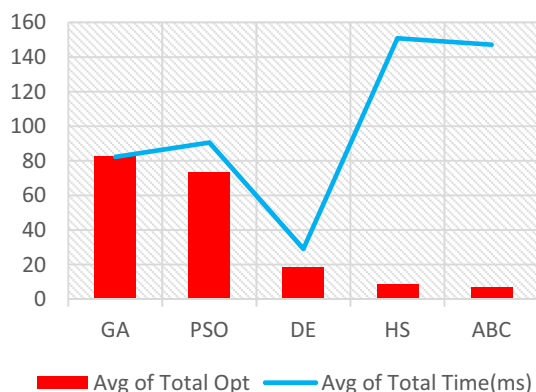
The fourth scenario:

we considered the application with 150 requests, which is located in the first layer of Fog. Figure 9 shows the comparison results algorithms for this scenario. The average amount of error reduction associated with the WCD metric with the GA is equal to 93.57, this value with the PSO algorithm is equal to 79.91, with the DE algorithm is equal to 17.06, with the HS algorithm is equal to 8.61 and with the ABC algorithm is equal to 4.52. As shown in Table 5, due to the low range of CPU resource requests in the third program, despite the high number of requests, the algorithms have shown better results than the second program scenario.

we considered the application with 200 requests in the smart home environment, which is in the first layer of Fog. Figure 10 shows the comparison results of algorithms for this scenario. The average amount of error reduction associated with the WCD metric with the genetic algorithm is equal to 92.59, this value with the PSO algorithm is equal to 73.18, with the DE algorithm equal to 15.84, with the HS algorithm equal to 7.32 and with the ABC algorithm is equal to 3.70. As shown, our proposed genetic algorithm has had better results than other algorithms when the number of requests is high. Table 6 is placed to show the comparison of comparative approaches in terms of average total execution time and the average amount of error reduction. Also, the information of Table 6 is depicted in Figs. 11 and 12 respectively by separating each scenario and each algorithm.

Table 6 The average amount of optimization and time from all application requests overall

Comparative algorithms	Avg. of total error reduction (Opt.)	Avg. of total time (ms)
GA	82.83	82.24
PSO	73.63	90.54
DE	18.62	29.14
HS	8.44	150.89
ABC	6.91	147.11

**Fig. 11** Average time of implementation in 4 scenarios for each algorithm (from Table 4)**Fig. 12** The total average error reduction (amount of optimization and time) from all application requests for each algorithm

Numerical results simulated in MATLAB simulator software for 40 times of four IoT applications with GA, PSO, DE, HS, and ABC algorithms (to optimal resource allocation for IoT devices in a fog environment with a machine learning approach) are given above.

In Fig. 11, the line graph illustrates the average time of implementation in 4 scenarios for each algorithm. From the given information in Table 4, in the first scenario, GA and PSO algorithms approximately have the same time, which is lower than the other algorithms in that scenario. Over the increasing number of requests, the execution time for GA and PSO climbed progressively from 18 and reached over 100 (ms) from the Application with 200 requests. However, it is yet less than HS and ABC algorithms. Additionally, the average time of GA is less than PSO. The HS algorithm has

had further execution time in the first place. Nevertheless, it has had a conflict with the ABC algorithm in the third scenario; thus, we can infer that the ABC algorithm witnessed the highest growth in the execution time of the program. Although the DE algorithm remained stagnant until the fourth scenario, when the number of requests increased it saw steady growth; afterward, it reaches approximately 39(ms). Totally, despite the DE algorithm has had a better execution time, it had not the best performance. Therefore, we cannot consider it as the proposed algorithm. Table 5 and Fig. 12 illustrate the overall average error reduction (value of optimization and time) among all application requests at each algorithm such as GA, PSO, DE, HS, and ABC. In terms of error reduction as the main goal of this optimization, the GA algorithm has had the highest amount in the total average through different situations. In addition, it would indicate the GA algorithm, also has the lowest execution time except for the execution time of the DE algorithm as compared to any other algorithms. Regardless of the change, with an enhancement of requests number, the execution time significantly increased in all the developed countries. In PSO, the execution time was ascending by about 90.54(ms) and the opposite 73.63 error reduction was descending. The GA showed 82.24(ms) of execution time with an 82.83 optimization value, which is the highest amount of decrement of error by any used algorithm. By the same token, the implementation time increased in the ABC and HS algorithm from 147.11 (ms) with a 6.91 error decrement, and 150.89(ms) with 8.44 error reduction respectively, and the error reduction parameter has not had acceptable results. Also, the execution time in DE algorithm only has had a little increase and reached 29.14(ms) with an 18.62 error reduction. In general, our proposed genetic algorithm has a better result and performance than other algorithms, which means that the GA algorithm provides fog resources more efficiently and with less error to IoT applications and has less delay, which brings the near to the desired quality of service.

7 Conclusion and future work

This paper utilized both meta-heuristic algorithms (customized GA) and machine learning clustering techniques to allocate requested resources of available fog nodes to IoT

applications efficiently. The Euclidean distance and optimization function is defined to optimally cluster resources which is tailored to resources requested by IoT applications. The error rates are representative of tolerable delay of time-sensitive IoT applications. To verify the proposed optimization model, four different scenarios in IoT applications for a smart home were conducted. The performance of customized GA was compared against PSO, DE, HS, and ABC meta-heuristic algorithms in the MATLAB simulator software environment. The numerical results show that our proposed GA algorithm, which uses machine learning techniques, has better results than other algorithms and has an overall 82.83 reduction in clustering error, which means that it reduces latency significantly in IoT applications. Another benefit is to increase the quality of services, which was also the purpose of this paper. As a limitation of the current work, this paper formulated a resource allocation problem to a single objective optimization which only pays on the users' desired QoS. In the future, we envisage to formulate resource allocation for dynamic and mobile ad-hoc devices which efficiently utilize fog computing platforms along with users' service level agreement simultaneously.

Author contributions Credit Contribution Statement Kimia Abedpour Formal analysis, Visualization Mirsaeid Hosseini Shirvani Conceptualization, Software Elmira Abedpour Investigation, Software

Funding Not applicable.

Data availability The data that support the findings of this study are available on request from the corresponding author. The data are not publicly available due to privacy or ethical restrictions.

Declarations

Conflict of interest The authors declare that there is no conflict of interest regarding the publication of this paper.

Ethical approval This article does not contain any studies with human participants or animals performed by any of the authors.

Informed consent Informed consent was obtained from all individual participants included in the study.

References

1. Qiu, J., Chen, Y., Tian, Z., Guizani, N., Du, X.: The security of internet of vehicles network: adversarial examples for trajectory mode detection. *IEEE Netw.* **35**(5), 279–283 (2021). <https://doi.org/10.1109/MNET.121.2000435>
2. Shayesteh, B., Hakami, V., Akbari, A.A.: Trust management scheme for IoT-enabled environmental health/accessibility monitoring services. *Int. J. Inf. Secur.* **19**, 93–110 (2020). <https://doi.org/10.1007/s10207-019-00446-x>
3. Brogi, A., Forti, A.: QoS-aware deployment of IoT applications through the fog. *IEEE Internet Things J.* **4**, 1185–1192 (2017). <https://doi.org/10.1109/JIOT.2017.2701408>
4. Hosseini Shirvani, M., Rahmani, A.M., Sahafi, A.: An iterative mathematical decision model for cloud migration: a cost and security risk approach. *Software* **48**(3), 449–485 (2018). <https://doi.org/10.1002/spe.2528>
5. Hosseini Shirvani, M.: To move or not to move: an iterative four-phase cloud adoption decision model for IT outsourcing based On TCO. *J. Soft Comput. Inf. Technol.* **9**(1), 7–17 (2020)
6. Asghari Alaie, Y., Hosseini Shirvani, M., Rahmani, A.M.: A hybrid bi-objective scheduling algorithm for execution of scientific workflows on cloud platforms with execution time and reliability approach. *J. Supercomput.* **79**, 1451–1503 (2023). <https://doi.org/10.1007/s11227-022-04703-0>
7. Hosseini Shirvani, M., Masdari, M.: A survey study on trust-based security in Internet of Things: challenges and issues. *Internet Things* **21**, 100640 (2023). <https://doi.org/10.1016/j.iot.2022.100640>
8. Cai, H., Xu, B., Jiang, L., Vasilakos, A.V.: IoT-based big data storage systems in cloud computing: perspectives and challenges. *IEEE Internet Things J.* **4**(1), 75–87 (2017). <https://doi.org/10.1109/JIOT.2016.2619369>
9. Gupta, H., VahidDastjerdi, A., Ghosh, S.K., Buyya, R.: iFogSim: a toolkit for modeling and simulation of resource management techniques in the Internet of Things, Edge and Fog computing environments. *Software* **47**(9), 1275–1296 (2017). <https://doi.org/10.1002/spe.2509>
10. Eliassen, A. T.: Poster Abstract: Data-centric IoT Services Provisioning in Fog-Cloud Computing Systems. *IEEE/ACM Second International Conference on Internet-of-Things Design and Implementation (IoTDI)*, 317–318 (2017).
11. Bonomi, F., Milito, R., Zhu, J., Addepalli, S.: Fog computing and its role in the internet of things. In: *MCC'12 - Proceedings of the 1st ACM mobile cloud computing workshop*, 13–15. (2012). <https://doi.org/10.1145/2342509.2342513>
12. Hong, C.-H., Varghese, B.: Resource management in fog/edge computing: a survey on architectures, infrastructure, and algorithms. *ACM Comput. Surv.* **52**(5), 1–37 (2019). <https://doi.org/10.1145/3326066>
13. Abbasi, M., Yaghoobikia, M., Rafiee, M., Jolfaei, A., Khosravi, M.R.: Efficient resource management and workload allocation in fog-cloud computing paradigm in IoT using learning classifier systems. *Comput. Commun.* (2020). <https://doi.org/10.1016/j.comcom.2020.02.017>
14. Rani, S., Ahmed, S.H., Rastogi, R.: Dynamic clustering approach based on wireless sensor networks genetic algorithm for IoT applications. *Wirel. Netw.* **2020**(26), 2307–2316 (2020). <https://doi.org/10.1007/s11276-019-02083-7>
15. Javadian Kootanaee, A., Poor Aghajan, A.A., Hosseini Shirvani, M.: A hybrid model based on machine learning and genetic algorithm for detecting fraud in financial statements. *J. Optim. Ind. Eng.* **14**(2), 169–186 (2021). <https://doi.org/10.22094/joie.2020.1877455.1685>
16. RonakSanjay: IJERT-survey on resource allocation strategies in cloud computing ER. *J. IJERT* (2013). <https://doi.org/10.14569/IJACSA.2012.030616>
17. Bonomi, F.: Connected vehicles, the internet of things, and fog computing. In: *The eighth ACMInt.workshop on vehicular inter-networking (VANET), Las Vegas, USA, September 19–23, pp. 13–15. ACM, New York, NY* (2011). <https://doi.org/10.1007/978-3-030-36167-9>
18. Ramzanpoor, Y., Hosseini Shirvani, M., Golsorkhtabaramiri, M.: Multi-objective fault-tolerant optimization algorithm for deployment of IoT applications on fog computing infrastructure.

- Complex Intell. Syst. **2022**(8), 361–392 (2022). <https://doi.org/10.1007/s40747-021-00368-z>
19. Zhou, Z., Gao, C., Xu, C., Zhang, Y., Member, S., Mumtaz, S., Member, S.: Social big-data-based content dissemination in internet of vehicles. *IEEE Trans. Ind. Inf.* **14**(2), 768–777 (2018). <https://doi.org/10.1109/TII.2017.2733001>
 20. Zhou, Z., Member, S., Liu, P., Feng, J., Zhang, Y., Member, S.: Computation resource allocation and task assignment optimization in vehicular fog computing: a contract-matching approach. *IEEE Trans. Veh. Technol.* **68**(4), 3113–3125 (2019). <https://doi.org/10.1109/TVT.2019.2894851>
 21. Ramzanpoor, Y., Hosseini Shirvani, M., Golsorkhtabaramiri, M.: Energy-aware and reliable service placement of IoT applications on fog computing platforms by utilizing whale optimization algorithm. *J. Adv. Comput. Eng. Technol.* **7**(1), 67–80 (2021)
 22. Taneja, M., Davy, A.: Resource-aware placement of IoT application modules in fog-cloud computing paradigm. In: *Proc. of the IFIP/IEEE symposium on integrated network and service management*, IM 15, IEEE, 1222–1228 (2017). <https://doi.org/10.23919/INM.2017.7987464>
 23. Azimi, S., Pahl, C., Hosseini, S.M.: Performance management in clustered edge architectures using particle swarm optimization. *Cloud Comput. Serv. Sci.* **2021**(1399), 233–257 (2021). https://doi.org/10.1007/978-3-030-72369-9_10
 24. Yin, L., Luo, J., Luo, H.: Tasks scheduling and resource allocation in fog computing based on containers for smart manufacturing. *IEEE Trans. Ind. Inf.* (2018). <https://doi.org/10.1109/TII.2018.2851241>
 25. Sinaga, K.P., Yang, M.: Unsupervised K-means clustering algorithm. *IEEE Access* **2020**(8), 80716–80727 (2020). <https://doi.org/10.1109/ACCESS.2020.2988796>
 26. Likas, A., Vlassis, N., Verbeek, J.J.: The global k-means clustering algorithm. *Pattern Recogn.* **36**(2), 451–461 (2003). [https://doi.org/10.1016/S0031-3203\(02\)00060-2](https://doi.org/10.1016/S0031-3203(02)00060-2)
 27. Singh, A.K., Dziuranski, P., Indrusiak, L.S.: Value and energy optimizing dynamic resource allocation in many-core HPC systems. In: *Proceedings-IEEE 7th international conference on cloud computing technology and science, CloudCom 2015*, 180–185 (2016). <https://doi.org/10.1109/CloudCom.2015.22>
 28. Gao, X., Liu, R., Kaushik, A.: Hierarchical multi-agent optimization for resource allocation in cloud computing. *IEEE Trans. Parallel Distrib. Syst.* **32**(3), 692–707 (2021). <https://doi.org/10.1109/TPDS.2020.3030920>
 29. Azimi S., Pahl C., Hosseini Shirvani, M.: Particle swarm optimization for performance management in multi-cluster IoT edge architectures. In: *Proceedings of the 10th International Conference on Cloud Computing and Services Science*, pp. 328–337. (2020). <https://doi.org/10.5220/0009391203280337>
 30. Devarasetty, P., Reddy, S.: Genetic algorithm for quality of service based resource allocation in cloud computing. *Evol. Intell.* **2021**(14), 381–387 (2019). <https://doi.org/10.1007/s12065-019-00233-6>
 31. Wu, C., Li, W., Wang, L., Zomaya, A.Y.: An evolutionary fuzzy scheduler for multi-objective resource allocation in fog computing. *Future Gener. Comput. Syst.* **117**, 498–509 (2021). <https://doi.org/10.1016/j.future.2020.12.019>
 32. Li, Q., Zhao, J., Gong, Y., Zhang, Q.: Energy-efficient computation offloading and resource allocation in fog computing for internet of everything. *China Commun.* **16**, 32–41 (2019). <https://doi.org/10.12676/j.cc.2019.03.004>
 33. Jia, B., Hu, H., Zeng, Y., Xu, T., Yang, Y.: Double-matching resource allocation strategy in fog computing networks based on cost efficiency. *J. Commun. Netw.* **20**(3), 237–246 (2018). <https://doi.org/10.1109/JCN.2018.000036>
 34. Mebrek, A., Merghem-boulahia, L., & Esseghir, M.: Efficient green solution for a balanced energy consumption and delay in the IoT-Fog-cloud computing. In: *2017 IEEE 16th International Symposium on Network Computing and Applications (NCA)*, Cambridge, MA, USA, 2017, pp. 1–4, <https://doi.org/10.1109/NCA.2017.8171359>
 35. Abbasi, M., Mohammadi Pasand, E., Khosravi, M.R.: Workload allocation in IoT-fog-cloud architecture using a multi-objective genetic algorithm. *J. Grid Comput.* **18**, 43–56 (2020). <https://doi.org/10.1007/s10723-020-09507-1>
 36. Pourkiani, M., & Abedi, M.: *Using machine learning for task distribution in fog-cloud scenarios: A deep performance analysis. international conference on information networking (ICOIN)*, Jeju Island, Korea (South), 2021, pp. 445–450 (2021). <https://doi.org/10.1109/ICOIN50884.2021.9333929>
 37. Tiso, J.: *Designing Cisco network service architectures (ARCH): foundation learning guide*. Cisco Press, A White Paper (2011).
 38. Shooshtarian, L., Lan, D., Taherkordi, A.: A clustering-based approach to efficient resource allocation in fog computing. In: *Communications in Computer and Information Science: Vol. 1080 CCIS* (pp. 207–224) (2019). https://doi.org/10.1007/978-3-030-30143-9_17
 39. Alenawy, T.A., Aydin, H.: Energy-aware task allocation for rate monotonic scheduling. *Proc. IEEE Real-Time Embed. Technol. Appl. Symp. RTAS.* (2005). <https://doi.org/10.1109/rtas.2005.20>
 40. Kennedy, J., Eberhart, R.: Particle swarm optimization. *Proc. IEEE Int. Conf. Neural Netw.* **4**, 1942–1948 (1995). <https://doi.org/10.1109/ICNN.1995.488968>
 41. Storn, R., Price, K.: Differential evolution-a simple and efficient heuristic for global optimization over continuous spaces. *J. Glob. Optim.* **11**, 341–359 (1997). <https://doi.org/10.1023/A:1008202821328>
 42. Yang, X.S.: Harmony search as a metaheuristic algorithm. *Stud. Comput. Intell.* **191**, 1–14 (2009). https://doi.org/10.1007/978-3-642-00185-7_1
 43. Karaboga, D., Gorkemli, B.: A quick artificial bee colony (qABC) algorithm and its performance on optimization problems. *Appl. Soft Comput.* **23**, 227–238 (2014). <https://doi.org/10.1016/j.asoc.2014.06.035>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Kimia Abedpour was born in 1998. She received her B.Sc. and M.Sc. degrees in the major of Computer Engineering-Software from Marlik Nowshahr Institute (Iran) in 2019 and Tabarestan Chalus Institute (Iran) in 2021, respectively. She is interested in pursuing a PhD degree in Computer Engineering. Her research interests are Cloud-Fog Computing and artificial intelligence. In her free time, she also enjoys experimenting with new programming

languages and technologies.



Mirsaeid Hosseini Shirvani received his B.Sc., M.Sc., and PhD all in Computer Software Engineering Systems at Universities in Tehran, IRAN. He has been teaching miscellaneous computer courses in several universities in Mazandaran province of IRAN since 2001. He was the former head of computer engineering department at IAU (Sari-Branch) during 2010-2012. Currently, he serves as Professor in Computer Engineering Department at IAU

(Sari-Branch). He also published several papers in authentic and world-reputed journals. Moreover, he serves as reviewer of numerous journals in IEEE, Elsevier, Wiley, Springer, Taylor & Francis, Emerald, etc. publications. His research interests are in the areas of cloud- and fog- computing, IoT, distributed systems, parallel processing, machine learning, and evolutionary computations.



Elmira Abedpour was born in 1996 in Iran. She received the B.Sc in Architectural Engineering from Azad university in Iran and she is studying for M.Sc in Digital skills for sustainable societal transitions in the Polytechnic university of Turin, Italy. Her favorite majors consist of data analyzing, programming, and AI.